

VISTRA — Final Unified Solution (SIH 2026 · PS 26126)

Team NEXORAS · Bharat Electronics Limited · Software · Smart Automation · prepared 3 Oct 2026

Label key: FACT = verified from a source · PROPOSED = our design choice · ESTIMATE = engineering approximation · TARGET = goal to be measured · MEASURED = real result (none exist yet; nothing below claims one).

1. EXECUTIVE SUMMARY

Final solution: VISTRA (Vision-based, Self-aware, Traversability- and Risk-aware Autonomy). It is a camera-first, GPS-denied UGV navigation stack with one thesis: *uncertainty in → safe behaviour out*. Segmentation doubt becomes map risk, pose doubt inflates that risk, a single weakest-link Navigation Confidence Score (NCS) caps speed and triggers recovery, and a time-to-collision (TTC) layer that includes measured latency and stopping time overrides everything. A live dashboard explains every slow, stop, replan and recovery in plain language.

What your six files actually contain. Three different ideas, not one: (a) the VISTRA blueprint and screening deck (uncertainty propagation, NCS governor, TTC, benchmark-first discipline); (b) VISTRA 2.0 (negative-obstacle reasoning, environmental self-awareness, mission-aware cost, energy-aware planning); (c) PRAGYA-Nav (the UGV *feels* terrain through slip, vibration and motor effort, learns from it, runs on Jetson with Nav2). They overlap heavily but contradict each other in places, and the current NEXORAS final deck is a half-merged mix.

The one addition I recommend. From PRAGYA-Nav I keep exactly one module, scoped small: **Experience-Calibrated Traversability (ECT)**. It attacks VISTRA's biggest admitted weakness (hand-set per-class traversability scores) by correcting those scores online from simulated slip, vibration and effort feedback, plus a short-lived terrain memory. Everything else in PRAGYA-Nav and VISTRA 2.0 that is not already in the blueprint is either folded in at near-zero cost or cut. You have told me scope creep is your known failure mode, so Sections 17 and 20 draw the freeze line explicitly.

Decisions on conflicts in your files

Conflict	Where it appears	Decision
Nav2 vs. self-written planners	PRAGYA deck, Marathi note, final NEXORAS deck list Nav2; blueprint says write D* Lite + DWA yourself	<i>Own D Lite + DWA.*</i> ROS 2 only for transport, sim bridge, rosbag. Nav2 appears only as the baseline.
Ubuntu 22.04 / Humble vs. 24.04 / Jazzy	Marathi note vs. blueprint	Jazzy + Harmonic on Ubuntu 24.04. FACT: ROS docs name Harmonic as Jazzy's recommended Gazebo release.
YOLOv8n-seg for terrain vs. SegFormer-B0	Marathi note vs. blueprint	SegFormer-B0 for terrain (dense per-pixel probabilities are needed for entropy). YOLO-nano only for moving objects.
Learns from physical driving, in a Software-category entry with no vehicle	PRAGYA deck, final NEXORAS deck	Implement ECT in simulation and say so plainly. Physical UGV stays optional validation.
Energy-aware planning	VISTRA 2.0 §11	CUT. Your own tier list files it under Advanced and the PS never asks for it.
Jetson + TensorRT in the stack	final NEXORAS deck	CUT from must-have. Mention only as a deployment path. The blueprint already lists TensorRT as cut.
6 terrain classes vs. 10	blueprint vs. Marathi note	7 classes: the blueprint's 6 plus soft/mud, since the slip story needs it. Check it maps onto RUGD/RELLIS labels before committing.

Conflict	Where it appears	Decision
Self-supervised novelty and patent angle	Marathi note	Downgrade the claim. TerraPN (IROS 2022) already learns terrain cost from IMU vibration and odometry error and extends DWA with it (FACT, arXiv 2202.12873). Our novelty is the integration plus the evidence, not the idea of learning from proprioception.

Defects in your current submission files (fix before upload)

- NEXORAS_SIH26126_final.pptx still says PRAGYA-Nav on slide 1 (title), slide 2 (the safest-path label and the comparison box) and slide 5 (heading sentence). Slide 3 lost the asterisk in D* Lite. Its tech stack still lists Nav2 and ONNX/TensorRT on Jetson, contradicting the blueprint.
- vistra_screening.pdf page numbers run 1, 2, 3, 5, 6, 6. Confirm nothing is missing.
- The metric targets in the Marathi note (85%+ completion, 90%+ recall, under 150 ms, under 300 ms safe-stop, 2-5% drift) have no sources. They are TARGET at best. Do not present them as expected results.

2. PROBLEM ANALYSIS

Layer	Analysis
Surface problem	PS 26126: GPS-denied outdoor UGV with camera as primary sensor. Path and hazard detection, visual localization, dynamic collision avoidance. Deliver perception AI + visual SLAM/odometry + a planner emitting wheel/motor commands. Success = collision-free A→B across outdoor scenarios.

Layer	Analysis
Root problem	Off-road navigation is not object detection. Unstructured ground has no lanes or curbs, looking flat is not the same as being drivable, and every model in the chain (segmenter, pose estimator) is silently wrong some of the time. Stacks that treat outputs as truth fail without warning.
Operational problem	Operators in search-and-rescue, agriculture or delivery need a vehicle that slows, asks for help or stops before a failure, and can say why.
Technical problem	(1) Monocular scale ambiguity and the flat-ground assumption in ground projection. (2) Ditches and negative obstacles are poorly detected by appearance. (3) Visual odometry degrades with blur, darkness, glare, low texture. (4) Latency eats stopping distance. (5) Sim-trained models do not transfer to real imagery.
Failure modes	Silent pose drift; unknown obstacle classed as free; full-speed driving with degraded vision; entering soft ground that looks like soil; oscillation between safe and unsafe states; demo crash on stage.
Opportunity	Make uncertainty a first-class quantity that the planner and the speed governor consume, then prove it with a reproducible adversarial benchmark instead of one demo.

3. EXISTING SOLUTIONS

Approach	Technology	Strength	Weakness	Gap
YOLO + ORB-SLAM + A* (typical SIH entry; your baseline)	Box detector, feature SLAM, grid A*	Easy to build	Detects only trained classes; binary occupancy hides doubt; silent SLAM drift; fixed-distance stop	No uncertainty, no confidence-aware behaviour
Segmentation + costmap	Per-pixel classes mapped to cost	Dense drivable-space estimate	Argmax discards confidence; class cost is a guess; same label can hide different traction	Label is not traversability
Nav2 with costmap layers	ROS 2 planners and controllers	Mature, configurable	Config-heavy; not uncertainty-aware by default; hard to explain	Behaviour not gated by perception or pose confidence
TerraPN (IROS 2022)	RGB patches + velocities in; IMU vibration and odometry error as self-supervised labels; DWA extension	Learns surface cost from robot-terrain interaction (FACT)	Needs a real moving vehicle; no pose-uncertainty or TTC safety layer	Closes the learning loop, not the safety loop

Approach	Technology	Strength	Weakness	Gap
WayFASTER	Image sequence + pose, temporal fusion, MPC	Predicts traversability beyond the current view	Heavier model, MPC tuning, no explicit confidence-gated behaviour	Same
LiDAR-centred stacks	3D geometry	Strong geometry, negative obstacles	Violates camera-primary; cost and power	Outside PS scope
End-to-end learned planning	Network from pixels to controls	Compact	Hard to verify or explain; data-hungry	Not defensible for safety
VISTRA blueprint / PRAGYA-Nav (ours)	See files	Complementary halves	Not yet one system	Merged here

Sources: TerraPN arXiv 2202.12873 (fetched this session). WayFASTER was seen only through a secondary summary, so read the arXiv paper before citing details. Other rows are PROPOSED characterisations of common practice, not benchmarked by us.

4. GAP ANALYSIS

1. **Uncertainty stops at the model output.** Existing stacks do not carry segmentation entropy and pose covariance into planning cost and speed.
2. **No single confidence authority.** Perception, localization and planning each have scores (or none), but nothing combines them into a behaviour decision with hysteresis.
3. **Stopping logic ignores latency.** Fixed-distance stop rules do not account for speed or measured pipeline delay.
4. **Class cost is hand-set and never corrected.** Learned-from-experience systems fix this but need hardware and lack the safety layer.
5. **Evidence is thin.** Typical entries show one run. Few ship a seeded scenario suite, a same-seed baseline, ablations and a judge-triggered fault injector.
6. **Failures are unexplained.** Why did it stop? has no answer in most stacks.

5. PROPOSED SOLUTION

Ten modules. Items marked ★ are the integrated novelty.

#	Module	Job
M1	Input Quality Monitor	Blur (Laplacian variance), brightness histogram, occlusion (frame variance), feature-count health
M2	Perception	SegFormer-B0 (7 classes) giving per-pixel probabilities; YOLO-nano for moving objects only
M3	★ Risk-Aware BEV Map	Expected traversability + entropy + geometry + proximity inflation, log-odds temporal fusion
M4	Localization	Stereo VO (RTAB-Map or ORB-SLAM3) + IMU/wheel EKF giving pose and covariance
M5	★ Experience-Calibrated Traversability	Online correction of class scores and a cell memory from slip, vibration, effort
M6	Planning	D* Lite (global) + DWA (local) on the risk map; mission cost = distance + terrain risk + localization risk + dynamic risk
M7	★ Confidence Engine + Behaviour Governor	NCS = weakest link; hysteresis FSM sets speed cap and mode
M8	★ TTC Safety Layer	Trajectory prediction, footprint vs. risk cells and predicted movers, latency-aware stopping time
M9	Recovery Manager	Slow, stop, relocalize, retrace, rotate-search, backtrack, safe stop
M10	Explainer + Dashboard + Fault Injector	Structured decision log; judge buttons for obstacle, blur, darken, cover camera

Target users: UGV operators, autonomy developers, safety and validation engineers.

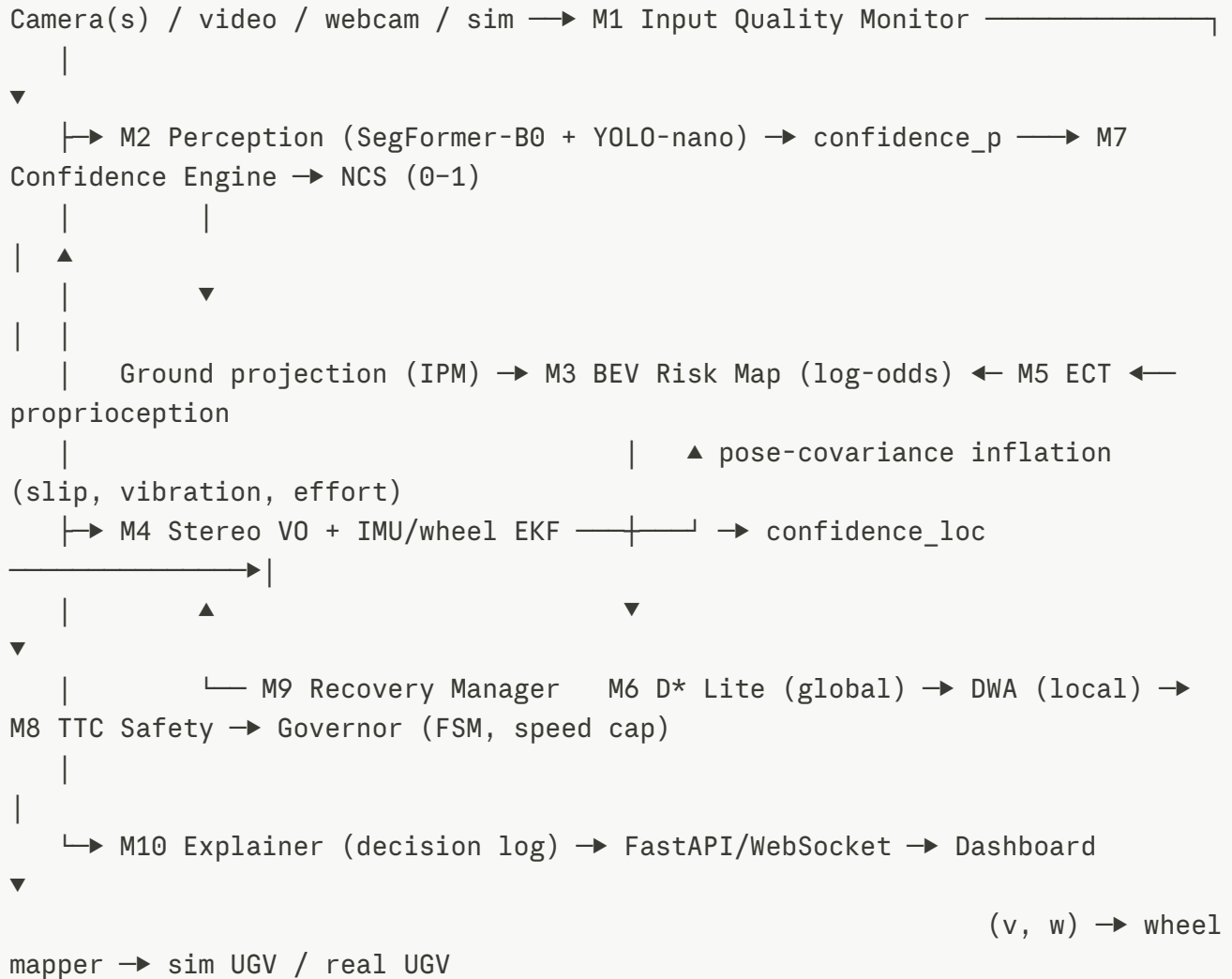
Operating environment: unstructured outdoor terrain (trail, grass, gravel, soft ground, rocks, ditches, slopes) with GPS unavailable.

6. CORE INNOVATION

Each item follows Existing → Limitation → Ours → Benefit, and says whether it is new in principle or new in combination.

1. **Uncertainty-propagating cost map.** Existing: argmax segmentation and binary obstacle boxes. Limitation: unknown hazards become free space. Ours: $R = (1-T) + \lambda_U \cdot U + \lambda_G \cdot G + \lambda_{\text{prox}} \cdot \text{inflation}$, with $T = \sum p_k \cdot t_k$ and U = normalised entropy, fused in log-odds and then inflated by pose covariance. Benefit: doubtful or unseen ground is costly by default. *Honest note:* entropy is also high at class boundaries, so narrow trails can look risky. Mitigation in Section 9 (calibration plus ablation).
2. **Single weakest-link confidence that drives behaviour.** Existing: full speed until failure. Ours: NCS = min of perception, localization, path, input-quality and geometry-agreement confidence, with hysteresis. Benefit: graded slowdown and recovery before a collision. *Old in principle; the evidence-backed integration is ours.*
3. **Latency-aware predictive safety.** Existing: stop if distance is under a threshold. Ours: $\tau_{\text{stop}} = t_{\text{latency}} + v/a_{\text{max}}$ compared with predicted TTC, with the prediction horizon scaled to the stopping time (Section 7 explains why this matters). Benefit: the stopping rule scales with speed and measured delay.
4. **Experience-Calibrated Traversability (ECT).** Existing: hand-set class scores, or proprioceptive learners such as TerraPN with no safety layer. Ours: class scores and a cell memory corrected online from slip, vibration and effort, feeding the same risk map and governor. Benefit: when the visual prior is wrong (a grass-looking patch that is actually soft), the vehicle learns once and slows or avoids next time. *Not new in principle (TerraPN); new as an integrated, confidence-gated, ablated component.*
5. **Judge-controlled adversarial evidence.** Existing: one scripted demo. Ours: fault-injection panel, same-seed side-by-side against the baseline, 20–30 seeded runs per scenario with confidence intervals, and ablations. Benefit: robustness is shown, not claimed.

7. SYSTEM ARCHITECTURE



Feedback loops (all PROPOSED): pose covariance inflates map cost; NCS caps speed; localization failure hands control to the Recovery Manager, which overrides the planner; planning failure triggers replan, then rotate-search, then safe stop; ECT updates class scores that feed the next map.

Block	Input	Processing	Output	Technology	Purpose
Hardware abstraction	Sim/video/webcam/robot streams	CameraInterface, SensorInterface, VehicleInterface	Uniform frames, IMU, wheel data	Python ABCs; ROS 2 topics	One core, four modes (sim, recorded, webcam, physical)

Block	Input	Processing	Output	Technology	Purpose
M1 Quality monitor	Frame	Laplacian variance, histogram, frame variance	input_quality $\in$ [0,1], flags	OpenCV	Detect blur, dark, cover before perception is trusted
M2 Perception	Frame	SegFormer-B0 $\rightarrow$ class probabilities; YOLO-nano $\rightarrow$ movers	p_k per pixel; tracked objects	PyTorch $\rightarrow$ ONNX Runtime	Dense drivable-space estimate plus moving objects
M3 Risk map	p_k, depth, pose, covariance	$T = \sum p_k t_k$; U = entropy; G geometry; project to BEV (20 m $\times$ 20 m at 0.2 m = 100 $\times$ 100 cells); log-odds fusion with decay	Risk grid + bands (green <0.25, yellow <0.5, orange <0.75, red $\geq$ 0.75)	NumPy	Turn doubt into cost
M4 Localization	Stereo frames, IMU, wheel odometry	VO (features, PnP, RANSAC) + EKF	Pose, covariance, confidence_loc	RTAB-Map or ORB-SLAM3 + robot_localization	GPS-free pose with honest uncertainty
M5 ECT	Class labels under footprint, slip, vibration, effort, NCS, pose confidence	Gated online update of t_k and a cell memory (Section 9)	Corrected t_k; memory layer	NumPy	Fix hand-set class scores from experience
M6 Planner	Risk grid, goal	D* Lite on coarse grid; DWA scoring goal progress, risk, smoothness, clearance	Path; (v, w) candidate	Python	Risk-aware route and local motion
M7 Confidence + governor	Component confidences	NCS = weakest link; hysteresis FSM	State, speed cap	Python	One authority for behaviour
M8 TTC layer	(v, w), footprint, risk grid, movers	Predict trajectory; TTC = first overlap; τ_{stop}	Collision Risk Score; override	Python	Hard safety net
M9 Recovery	Failure signals	Scripted responses (Section 15)	Overrides to planner	Python FSM	Fail safe, not fail silent
M10 Explainer	Every state change	Structured log with reason	Dashboard feed	FastAPI + WebSocket	Explainability and evidence
Wheel mapper	(v, w)	$v_{left} = v - w \cdot L/2$; $v_{right} = v + w \cdot L/2$, clamped	Motor commands	Python	PS deliverable: wheel/motor commands

Prediction horizon correction (a flaw I found in the blueprint). The blueprint predicts 3 s ahead but continues only if $TTC > 3 \times \tau_{stop}$. With $\tau_{stop} = t_{latency} + v/a_{max}$, any τ_{stop} above 1 s makes $3 \times \tau_{stop}$ exceed 3 s, so the continue branch becomes unreachable at higher speed. **Fix (PROPOSED):** horizon $H = \max(3 \text{ s}, 3 \times \tau_{stop} + 0.5 \text{ s})$, and the speed cap

must keep τ_{stop} low enough for the sensing range. *Illustration only, assumed numbers:* $v = 1.0$ m/s, $a_{\text{max}} = 1.5$ m/s², latency 0.15 s gives $\tau_{\text{stop}} \approx 0.82$ s ($H = 3$ s is enough). At $v = 2.0$ m/s, $\tau_{\text{stop}} \approx 1.48$ s, so $H \approx 4.9$ s.

8. COMPLETE DATA FLOW

1. **Acquire** (30 Hz target, PROPOSED): stereo frames + simulated IMU (≈ 100 Hz) + wheel odometry (≈ 50 Hz), timestamped on one clock.
2. **Quality check (M1)**: blur, brightness, occlusion $\rightarrow$ input_quality. Frames below threshold raise flags to M7 and M9.
3. **Perceive (M2)**: resize, run SegFormer-B0 through ONNX Runtime $\rightarrow$ p_k per pixel; run YOLO-nano at lower rate for people, animals, vehicles. Compute per-pixel max-probability and entropy.
4. **Localize (M4, parallel)**: stereo VO tracks features $\rightarrow$ pose; EKF fuses IMU/wheel; output pose, covariance trace, inlier ratio, feature count $\rightarrow$ confidence_loc.
5. **Project (M3)**: ground-plane projection with known camera height, pitch and IMU roll/pitch compensation; stereo depth adds the geometry term G (and a below-ground-plane test for ditches in sim).
6. **Fuse (M3)**: motion-compensate the previous grid, update log-odds with decay, inflate costs by pose covariance, compute bands.
7. **Calibrate (M5)**: if NCS and localization are healthy, update class scores and cell memory from the proprioceptive experience of cells just driven over.
8. **Plan (M6)**: D* Lite replans on map changes; DWA scores candidate (v, w) on the risk map.
9. **Gate (M7)**: compute NCS; the FSM sets the mode and speed cap with hysteresis.
10. **Verify (M8)**: predict the trajectory over horizon H , compute TTC and τ_{stop} (with measured latency), choose continue / slow / brake / emergency stop.
11. **Act**: map (v, w) to left/right wheel speeds; send to the vehicle interface.
12. **Explain and record (M10)**: every state change writes a structured reason; stream to the dashboard; log to rosbag/SQLite for replay and benchmarking.

Local vs. cloud: **everything in steps 1–11 runs on one machine with no network**. Only the dashboard and logs may be remote, and the vehicle never depends on them.

9. AI/ML ARCHITECTURE

Why AI is needed. Dense terrain labelling in unstructured scenes cannot be written as rules; a learned segmenter is the only practical way to get per-pixel class probabilities. Everything downstream of the segmenter is deliberately classical and explainable.

Aspect	Design
Model	SegFormer-B0, FACT: about 3.7–3.8 M parameters (arXiv 2105.15203), fine-tuned from a public checkpoint. CPU-only fallback: PIDNet-S or BiSeNetV2. Moving objects: YOLOv8n/YOLO11n.
Input	RGB frame at reduced resolution (choose from measured FPS).
Output	Per-pixel probabilities over 7 classes: smooth-drivable, rough-drivable, soft/mud, vegetation (non-traversable), hard obstacle, water/ditch/negative, background/sky.
Training data	Public: RUGD, RELLIS-3D (from your references; map labels to 7 classes and verify the mapping), Freiburg Forest, GOOSE if accessible. Synthetic: auto-labelled sim frames across lighting. Self-collected: about 300–500 campus/trail frames (ESTIMATE of effort) for validation. Splits by sequence or location, never random frames; keep a held-out real set that never enters training.
Training method	Fine-tune on mixed real + sim with augmentation (brightness, blur, shadow, jitter, occlusion patches). Report per-class IoU, with ditch and obstacle reported separately.
Edge vs. cloud	Inference on-device via ONNX Runtime. Training on Colab or a GPU box.

Aspect	Design
Confidence	Max-probability and normalised entropy per pixel. Softmax is often over-confident under domain shift (known behaviour of neural classifiers, not a SegFormer-specific measurement), so apply temperature scaling on a held-out set and report expected calibration error (ECE) . Without ECE, do not call the entropy calibrated.
False positives (phantom obstacle)	Temporal log-odds fusion with decay filters flicker; hysteresis in the FSM; replan before stopping.
False negatives (missed hazard)	Free-space-first formulation: low-confidence ground is risky, not free. Geometry term as a second signal. Ditch recall expected to be poor on appearance alone (blueprint admits this); report it openly.
Boundary-entropy problem	Entropy peaks at class edges. Mitigation (PROPOSED): weight U by local max-probability, apply it to cells not just pixels, tune λ_u , and run the entropy-off ablation to show the net effect on trails.
Model update	Offline re-fine-tune between runs. Online change is limited to ECT's class scores, which are numbers, not network weights.
Explainability	Overlay of class and entropy; per-cell risk decomposition (1-T, U, G, inflation); decision log with reasons.

ECT specification (the one learning loop; PROPOSED, deliberately lean).

- Experience scalar per driven cell: $e = \text{clip}(a \cdot \text{slip} + b \cdot \text{vib_norm} + c \cdot \text{odom_disagreement} + d \cdot \text{effort_norm}, 0, 1)$. Weights a-d are tuned in sim and reported.
- Class update: $t_k \leftarrow (1-\eta) \cdot t_k + \eta \cdot (1-e)$, with small η (about 0.05), clamped to a floor and ceiling so one bad event cannot zero a class. Update only when the classifier was confident on that cell, NCS is at least CAUTION, and localization is healthy; otherwise the experience is attributed to the wrong cell.
- Cell memory: remember high-e cells with decay so a slippery patch is avoided on the next pass.

- **Sim design to avoid circular learning:** randomise hidden friction per patch so that visual class and true traction disagree for some patches. Then the loop is shown correcting a wrong prior, not rediscovering a number we typed in.
- **Honest scope:** this is online calibration of class scores, not a deep self-supervised network. Patch-appearance regression (TerraPN-style) stays in Future.
- **Cut-rule:** if ECT is not passing its tests by the end of week 6, drop it to a Future-work slide. The rest of VISTRA stands without it.

NCS (confidence engine). NCS = min over { perception (mean max-probability in the path corridor), localization (tracked features vs. expected, RANSAC inlier ratio, reprojection error, EKF covariance trace), path (cost margin vs. alternatives), input quality, geometry agreement }. Weakness (mine): components come from different sources and are not commensurable, and one noisy component can pin the vehicle in CAUTION. Mitigation: map each component to [0,1] with thresholds chosen from fault-injection data, smooth with an exponential moving average, and measure the completion-time cost of the governor in the benchmark.

NCS	State	Behaviour
above 0.7	NORMAL	Full speed cap
0.4-0.7	CAUTION	Speed cap scales down; wider inflation
0.2-0.4	RECOVER	Stop or creep; relocalize; re-observe
below 0.2	SAFE_STOP	Hold; alert

Thresholds are starting values (from the blueprint) to be tuned from seeded runs; use different enter and exit thresholds for hysteresis.

10. SENSOR / HARDWARE ARCHITECTURE

The PS is Software category, so the **primary prototype has no physical hardware**. Sensors are simulated; the same interfaces accept real ones later.

Sensor	Why needed	Limitation	If it fails
Stereo camera (primary)	Perception, VO, metric depth for geometry and negative obstacles	Blur, glare, darkness, low texture; depth noise at range	M1 flags; NCS drops; slow, then recover or stop
IMU	Roll/pitch for ground projection; bridges short visual gaps; vibration for ECT	Drift; cannot localize alone	EKF covariance grows; planner inflates costs; timeout forces safe stop
Wheel encoders	Odometry; slip estimate by comparing with VO	Wheel slip corrupts it (which is also the ECT signal)	Visual-only pose; report drift
Motor current/effort (sim, optional real)	ECT effort term	Needs motor driver telemetry	ECT skips the term

Positioning to judges (consistent with the PS wording): camera is the primary perception and localization sensor. IMU and wheels bridge short dropouts and supply the learning signal; a camera-only ablation reports the drift difference.

Optional physical validation hardware is costed in Section 19 and is attempted only if every core milestone is done.

11. COMMUNICATION ARCHITECTURE

Channel	Protocol	Rate	Purpose
Sim/robot ↔ core	ROS 2 topics (DDS)	camera 30 Hz, IMU about 100 Hz, wheels about 50 Hz (PROPOSED)	Sensor ingress; wheel-command egress
Core ↔ API	In-process or local ROS bridge	event driven	Hand state to the web layer

Channel	Protocol	Rate	Purpose
API ↔ dashboard	WebSocket, JSON	state 10 Hz; map as compressed grid 2-5 Hz; log events on change	Live view and decision feed
Dashboard → core	WebSocket commands	on click	Fault injection, mission goal, e-stop
Logging	rosbag2 / SQLite	continuous	Replay, benchmarking, backup demo

Rules: autonomy never waits on the dashboard; if the socket drops, the vehicle keeps its current safe behaviour and the dashboard reconnects and resynchronises from the log. Dashboard e-stop is advisory in sim; on a real vehicle a hardware e-stop is mandatory.

12. SOFTWARE ARCHITECTURE

Layer	Choice	Why
Frontend	React + TypeScript, 2D canvas/SVG	Blueprint cut Three.js; canvas is enough and cheap
Backend	FastAPI + WebSocket	Simple async real-time push
AI/ML	PyTorch (train), Hugging Face Transformers (SegFormer fine-tuning), ONNX Runtime (inference), OpenCV	Standard, CPU-friendly deployment
Robotics	ROS 2 Jazzy, Gazebo Harmonic (FACT: Harmonic is the recommended pairing for Jazzy on Ubuntu 24.04), robot_localization	Transport, sim bridge, EKF
Localization	RTAB-Map or ORB-SLAM3, decide once; own OpenCV VO + EKF as fallback; time-box integration to one week	SLAM integration is the top schedule risk in your register
Planning	Own D* Lite + DWA	Explainable, tunable, no Nav2 configuration burden

Layer	Choice	Why
Database	SQLite plus rosbag2	Local, zero-ops, replayable
Deployment of MVP	Static React build (Vercel) fed by recorded or in-browser sim state	Judges can open it without installing ROS

Dual-fidelity simulation (design change vs. blueprint, PROPOSED). The blueprint's protocol is 20–30 seeded runs × 10 scenarios × (baseline + VISTRA + 3 ablations) = roughly 1,000–1,500 runs. At about 2 minutes per Gazebo run that is on the order of 33–50 hours of sequential wall-clock (ESTIMATE). Use two tiers:

- **Tier 1, fast headless sim** (Python 2.5D kinematic world with a modelled perception-noise layer and the real planner, governor, TTC and recovery code): runs the full statistical benchmark in minutes to hours. Claims from this tier are about **decision logic**, never about perception quality.
- **Tier 2, Gazebo Harmonic with the real perception and VO in the loop:** fewer seeds (for example 5) on four showcase scenarios. Claims about perception come from Tier 2 and from the real-video track (RUGD/RELLIS plus your campus clips), which is open-loop and must be labelled so.

This split is what lets the numbers be statistically honest and the demo still show pixels to wheels. State the split in the deck and to judges.

13. DIGITAL TWIN / SIMULATION

The simulator is the twin: a parameterised, seeded world that stands in for the outdoor site.

Element	Implementation
Real-world environment	Trail, grass, gravel, soft/mud patch, rocks, trees, a ditch, slopes, a moving obstacle, lighting variants. Real footage (RUGD/RELLIS plus own clips) stands in for the physical site in the open-loop track.
Virtual representation	Gazebo Harmonic world (Tier 2) and a fast Python world (Tier 1), both generated from the same scenario file and seed (obstacle positions, lighting, start/goal, hidden friction per patch).

Element	Implementation
Live sensor data	Simulated stereo, IMU, wheel odometry published as ROS 2 topics.
AI prediction and system state	Segmentation overlay, BEV risk map, pose with covariance, NCS and state, TTC.
Predicted future state	Predicted trajectory and moving-obstacle positions over horizon H, drawn on the map.
History	rosvbag2 and SQLite; any run replays deterministically for the backup demo.

Physical and digital sides talk through the VehicleInterface and CameraInterface only. Swapping SimVehicle for RealVehicle must not change navigation code. Test that claim once in the schedule, otherwise it is an assumption.

14. DASHBOARD

One screen, four zones (2D canvas, no 3D).

- **Top bar (Overview):** mission status, NORMAL/CAUTION/RECOVER/SAFE_STOP badge, NCS gauge with its five components, e-stop state, FPS and measured latency.
- **Left (Live monitoring):** camera feed with segmentation and entropy overlay toggle; below it the wheel panel (v, w, left/right speed).
- **Centre:** top-down BEV risk map (green to red bands), planned path, driven path, UGV pose with covariance ellipse, detected obstacles, ECT-flagged cells, replan markers.
- **Right (AI and analytics):** TTC and τ_{stop} gauge, collision risk score, explanation feed (for example: $t=42.1$ s CAUTION: localization confidence 0.41, features 38→12, speed cap 0.8→0.3 m/s), run summary (collisions, replans, recovery time).
- **Bottom (Emergency and control):** fault-injection buttons (spawn obstacle, blur, darken, cover camera, degrade localization), side-by-side toggle (baseline vs. VISTRA, same seed), playback controls for the backup mode.
- **Critical alert state:** when SAFE_STOP or emergency stop fires, the banner shows location, cause, recommended action and response status.

15. FAILURE-SAFE SYSTEM

Every state transition writes a reason to the decision log. No silent failures.

Failure	Detection	Backup	Safe state	Recovery
Poor light or shadows	Brightness histogram, entropy spike	Lean on IMU/wheel, inflate costs	CAUTION, reduced speed	Resume when NCS clears the exit threshold
Motion blur	Laplacian variance drop	Reduce speed until sharp	CAUTION	Sharpness returns
Camera obstruction or input loss	Frame variance collapse; frame timeout	None (camera is primary)	Stop after short grace period, alert	Frames return, then re-observe
Featureless terrain	Tracked features below minimum	Dead-reckon with covariance growth	Slow, then stop on timeout	Features return
Localization failure	Covariance or inlier collapse	Relocalize against recent keyframes	RECOVER: stop/creep, rotate slowly	Retrace toward last good keyframe; resume when confidence restores; else safe stop
Sudden or moving obstacle	TTC drop	Replan, evade	Brake or emergency stop per TTC bands	Path clear
Planning failure or dead end	No path; repeated replans without progress	Rotate-search, back up, backtrack to last junction and mark region blocked	Hold	New route found

Failure	Detection	Backup	Safe state	Recovery
Compute overload or latency spike	Cycle-time watchdog	Lower resolution; derate speed (τ_{stop} grows with latency)	Slow or stop	Latency back in budget
Perception wrong (out-of-distribution)	Low max-probability, high entropy, sim-vs-geometry disagreement	Treat as risky	CAUTION or RECOVER	Confidence returns
IMU or wheel sensor failure	Stale or implausible data	Visual-only pose with wider covariance	CAUTION, more conservative	Sensor returns
Dashboard or network loss	Socket timeout	None needed; autonomy is local	Unchanged	Reconnect and resync from log
Power or hardware fault (real vehicle)	Undervoltage, watchdog	Hardware e-stop, independent of software	Motors off	Operator restart
Data corruption or tampering	Log checksums; signed model file	Reload known-good model	Refuse to start on mismatch	Re-deploy

GPS is not a failure mode: the system never relies on it.

16. CYBERSECURITY

Prototype (PROPOSED): dashboard served on localhost or behind a token; WebSocket commands require a session token and are rate-limited; fault-injection commands exist only in sim mode; model files checked by hash; no secrets in the repo; dependencies pinned.

Real deployment (PROPOSED, beyond hackathon scope): authenticated and encrypted command channel (mutual TLS or equivalent), per-device identity, signed firmware and model updates, secure boot, role-based access (operator vs. engineer), append-only audit

log, and a hardware e-stop that no network path can override. Camera-first navigation also has an adversarial-input surface (glare, spoofed patterns); the confidence engine plus geometry cross-check is the main defence, and this is untested.

17. PROTOTYPE: WHAT TO BUILD

MUST HAVE (demo cannot work without these)

- Hardware abstraction + sim vehicle; seeded scenario generator.
- SegFormer-B0 (ONNX) with measured FPS and latency; 7-class output.
- BEV risk map with bands and temporal fusion; pose-covariance inflation.
- VO + EKF (or own OpenCV fallback) with ATE logged against sim ground truth.
- D* Lite + DWA driving $A \rightarrow B$; wheel-command mapper.
- TTC safety layer with latency-aware τ_{stop} .
- Dashboard with all panels live; baseline (YOLO + same VO + A* + fixed-distance stop) on identical seeds; benchmark harness.

SHOULD HAVE

- NCS, hysteresis FSM, Recovery Manager, explanation feed, fault injector.
- ECT (class-score calibration + cell memory) in sim, with the hidden-friction scenario.
- Negative-obstacle handling via the geometry term (below-ground-plane cells from stereo in sim), reported as its own recall number.

FUTURE (say so in the deck, do not build) Physical UGV on Jetson/Pi, TensorRT, patch-appearance regression for ECT, learned obstacle-motion prediction, energy-aware planning, multi-UGV shared maps, thermal/radar fallback.

Scope freeze: end of week 6. Weeks 7–10 are for testing, benchmarking and rehearsal only. New ideas after the freeze go on the Future slide.

18. LIVE DEMO

1. **Normal run.** No GPS shown. Camera feed and an empty map. Mission $A \rightarrow B$ starts; segmentation overlay and BEV risk map build up; global path appears.
2. **Judge spawns an obstacle.** TTC drops, τ_{stop} line moves, speed falls, replan fires, new path drawn, vehicle follows it. Decision feed states each reason.

3. **Judge darkens or blurs the camera.** Input quality and NCS fall; CAUTION then RECOVER; vehicle slows, relocalizes, resumes. Pose covariance ellipse visibly grows then shrinks.
4. **Judge covers the camera.** Immediate stop, alert, no uncontrolled motion; uncover to resume.
5. **Ditch case.** Vehicle approaches the ditch; geometry and uncertainty raise risk; route avoids it. Report negative-obstacle recall honestly.
6. **Learn and remember (ECT).** The vehicle crosses a grass-looking patch with hidden low friction, slips, flags the cells and lowers that class score; on the second pass it slows or detours. State clearly that this is simulated physics.
7. **Shortest vs. safest.** Show a case where the shortest route crosses high-risk ground and VISTRA takes the longer, safer one.
8. **Finish.** Summary panel: collisions, replans, recovery time, clearance.
9. **Same seed, baseline.** Play the baseline run side by side.
10. **Backup.** Pre-recorded video and rosbag replays of every scenario, rehearsed at least three times.

19. BILL OF MATERIALS

Prototype (software-only) — ESTIMATE, verify current prices.

Item	Purpose	Spec	Qty	Approx. cost
Development laptop (existing)	Sim, inference, dashboard	16 GB RAM, Ubuntu 24.04 (dual-boot or native); GPU helpful	1	existing
Cloud GPU (free tier)	Fine-tuning SegFormer-B0	Colab or similar	1	₹0-2,000
Phone camera	Own campus/trail clips for validation	Any recent phone	1	existing
Storage	Datasets, rosbags	256 GB+ SSD	1	₹0-4,000

Optional physical validation (only after core milestones) — ESTIMATE ranges, not quotes.

Item	Purpose	Spec	Qty	Approx. cost
Differential-drive chassis + motors	Platform	4WD skid-steer kit	1	₹6,000–15,000
Motor driver + encoders	Control, slip estimate	e.g. BTS7960 class + quadrature encoders	1 set	₹3,000–6,000
Stereo/depth camera	Primary sensor	OAK-D Lite or RealSense class	1	₹12,000–25,000
IMU	Orientation, vibration	BNO085 or ICM-20948 class	1	₹1,500–3,500
Edge computer	On-board inference	Raspberry Pi 5 or Jetson Orin Nano class	1	₹10,000–35,000
Battery, e-stop, wiring	Power and safety	LiPo pack, hardware e-stop switch	1 set	₹3,000–8,000

Total optional hardware: roughly ₹35,000–90,000 (ESTIMATE). Real-deployment hardware would add ruggedised enclosure, redundant sensing and certified safety components; not costed here.

20. IMPLEMENTATION ROADMAP AND TEAM

Assumes 8–10 weeks as in the blueprint. **Compress to your real finale date; I do not know it.**

Phase	Weeks	Definition of done	Lead
1. Sim + virtual UGV + scenario seeds + baseline harness	1–3	Teleop drive, topics streaming, seeded worlds, baseline running end-to-end	Sim/integration
2. Perception	1–3	SegFormer-B0 ONNX, 7-class IoU on real and sim, FPS and latency measured	Perception

Phase	Weeks	Definition of done	Lead
3. Risk map	2-3	BEV bands, temporal fusion, geometry term	Planning/safety
4. Localization	2-4	VO + EKF, ATE vs. ground truth, one-week time-box	Localization
5. Planning	3-5	D* Lite + DWA drive A→B on a clear world	Planning/safety
6. TTC layer	5-6	Stops before every injected obstacle across 20 seeds	Planning/safety
7. NCS + governor + Recovery Manager	5-6	Hysteresis verified; recovers from injected blur, dark, loss	Localization + planning
8. ECT	5-6 (hard stop)	Hidden-friction scenario shows lower slip on second pass, or cut	Perception + planning
9. Dashboard + fault injector	3-8 (parallel)	All panels live from WebSocket	Dashboard/benchmark
10. Benchmark + ablations	7-9	Baseline vs. VISTRA on all scenarios; entropy-off, governor-off, TTC-off, ECT-off	Dashboard/benchmark
11. Final demo	9-10	Three full rehearsals including backup mode	Everyone

Suggested roles for five people plus a sixth: perception lead (datasets, SegFormer, ONNX, class mapping); localization lead (VO/SLAM, EKF, confidence signals, relocalization); planning/safety lead (risk map, D* Lite, DWA, TTC, FSM, ECT); sim/integration lead (Gazebo worlds, scenario generator, ROS 2 bridge, Docker image, rosbag backup); dashboard and benchmark lead (React, FastAPI, harness, plots, fault injector); sixth: baseline, documentation, deck and rehearsal.

21. PERFORMANCE METRICS

KPI	How measured	TARGET (PROPOSED)	MEASURED
Mission success rate	Seeded runs per scenario, mean and 95% CI	Not lower than baseline in any scenario; higher where perception is degraded	not yet measured
Collision rate and minimum clearance	Sim ground truth	Lower than baseline with non-overlapping CIs on unknown-obstacle and poor-light scenarios	not yet measured
False-stop rate	Stops with no true hazard	Reported alongside collision rate; must not be traded away silently	not yet measured
Per-class IoU (ditch and obstacle separately)	Held-out real set split by sequence	Report; no target for ditch recall (expected weak)	not yet measured
Segmentation FPS and latency	Your hardware, end to end	At least 10 FPS on a CPU-only laptop at the chosen resolution; end-to-end under 150 ms	not yet measured
Localization error (ATE)	Against sim ground truth, drift per distance	Under 5% of distance travelled in Tier 2; camera-only ablation reported	not yet measured
Recovery success and time	Injected blur, dark, loss, cover	Recovers in injected cases or stops safely in all	not yet measured
Calibration (ECE) of segmentation	Held-out set before and after temperature scaling	Lower after scaling	not yet measured
ECT effect	Slip or interventions, first vs. second pass	Lower on second pass in hidden-friction scenario	not yet measured

KPI	How measured	TARGET (PROPOSED)	MEASURED
Governor overhead	Mission time with vs. without governor	Reported honestly	not yet measured

The numeric targets above are my proposals, not facts. Replace them with what you measure; if the baseline wins a scenario, say so and explain why.

22. SCALABILITY

Scale	What changes
1 device	Everything on-board; dashboard on a laptop.
10 devices	Per-vehicle identity and config; one operator console showing many NCS/state badges; central log store; shared scenario files for regression.
100 devices	Fleet telemetry broker (message queue), time-series database, model registry with signed versions and staged roll-out, over-the-air updates with rollback, remote diagnostics from decision logs.
1,000+ devices	Multi-site cloud backend, anonymised terrain-experience aggregation across vehicles (a Future idea needing privacy and validation work), automated regression on every release, device-management and security operations.

On-vehicle autonomy never depends on the cloud at any scale. Multi-UGV shared maps and federated terrain learning are Future items and are not demonstrated.

23. IMPACT

No outcome below is claimed as achieved. These are the benefits the benchmark is designed to measure.

- **Safety:** fewer collisions and larger minimum clearance than a conventional detector-plus-A* stack on the same seeds (to be measured).

- **Reliability under degradation:** recovery or safe stop in every injected blur, darkness, cover and localization fault (to be measured).
- **Transparency:** every slow, stop and replan has a logged, human-readable reason, which supports operator trust and post-incident review.
- **Low cost:** camera-first design avoids LiDAR cost and power; software-only prototype costs close to nothing.
- **Use cases named in the PS:** search-and-rescue, agriculture, delivery. Mines, solar farms and defence logistics appear in your drafts; keep them as examples, not claims.

24. RISKS AND MITIGATION

Risk	Likelihood / impact (blueprint, ESTIMATE)	Mitigation
SLAM integration eats weeks	High / High	One-week time-box, own OpenCV VO + EKF fallback, decide library once
Sim-to-real perception gap	High / Medium	Fine-tune on sim + real mix; separate open-loop real-video track; state it openly
Weak ditch and negative-obstacle recall	High / Medium	Uncertainty + stereo-geometry term; report recall separately
Live sim crashes at demo	Medium / High	Rosbag replays, pre-rendered video, playback dashboard, three rehearsals
Gazebo/ROS setup pain on laptops	Medium / Medium	One Docker/Ubuntu image; Webots fallback
Latency too high for TTC claims	Medium / Medium	Lower resolution, ONNX Runtime, measured τ_{stop}
Scope creep (now includes ECT)	Very high / High	Freeze at end of week 6; ECT cut-rule; Future slide for everything else

Risk	Likelihood / impact (blueprint, ESTIMATE)	Mitigation
Metric overfitting	Medium / High	Sequence-level splits; held-out real set; tune on a separate seed set
Benchmark takes too long in Gazebo	High / High (my addition)	Two-tier sim (Section 12)
Entropy inflates risk on narrow trails	Medium / Medium (my addition)	Boundary-aware weighting, tune λ_u , entropy-off ablation
NCS min-rule pins vehicle in CAUTION	Medium / Medium (my addition)	Calibrate components from fault data, smoothing, report governor overhead
ECT learns what we typed into the simulator	Medium / Medium (my addition)	Hidden per-patch friction independent of visual class; label as simulated
YOLO licence terms for any real deployment	Unverified	Check the Ultralytics licence before commercial or BEL use; keep detector swappable
Deck and docs diverge again	High / Medium (my addition)	One source-of-truth document (this one); regenerate the deck from it

25. RED-TEAM ANALYSIS

- *Skeptical judge: isn't this YOLO + SLAM + A with extra steps?** Our baseline is exactly that and is run on identical seeds. The difference is carried by three ablations. If an ablation shows no gain, we say so.
- **Domain expert: your TTC horizon is too short at speed.** Correct in the blueprint; fixed by $H = \max(3 \text{ s}, 3 \times \tau_{\text{stop}} + 0.5 \text{ s})$. Also check that the IPM flat-ground assumption is compensated with IMU pitch/roll and that far cells are down-weighted.
- **Hardware engineer: where is the vehicle?** There is none in the core plan. Real slip, vibration and effort are noisier than simulated ones; ECT is therefore sim-validated and hardware-pending. We say that first.

- **AI researcher: softmax entropy is not true uncertainty.** Agreed. We apply temperature scaling, report ECE, and treat entropy as a cheap proxy with a measured ablation. Ensemble or test-time augmentation is a Future upgrade.
- **Government or industry stakeholder: how does this get certified for use?** It does not, at this stage. The route is staged: sim evidence, then instrumented field trials with a hardware e-stop, then safety-case work. We make no certification claim.
- **Competitor: they add a LiDAR or a bigger model and win on accuracy.** They may on raw perception, but they leave camera-first, cost and explainability. We win only if our benchmark is better on safety under degradation, so that is where effort goes.

Improvements already made from this exercise: horizon fix, dual-tier simulation, hidden-friction ECT scenario, ECE reporting, ECT cut-rule, honest novelty wording relative to TerraPN.

26. EVIDENCE MATRIX

Claim	Evidence	Source	Confidence
SegFormer-B0 has about 3.7–3.8 M parameters	Paper figure and table	arXiv 2105.15203 (fetched)	High
Harmonic is the recommended Gazebo release for ROS 2 Jazzy (Ubuntu 24.04)	Release notes	docs.ros.org Jazzy release page; Gazebo getting-started page (fetched)	High
Learning terrain cost from IMU vibration and odometry error, with a DWA extension, is published prior art	TerraPN abstract	arXiv 2202.12873 (fetched)	High
WayFASTER predicts traversability self-supervised with temporal fusion and MPC	Secondary summary only	Review site; read arXiv before citing	Medium-low

Claim	Evidence	Source	Confidence
D* Lite supports incremental replanning	Your reference list	Koenig & Likhachev 2002, AAAI	Medium (not re-fetched)
DWA is a standard local collision-avoidance method	Your reference list	Fox, Burgard & Thrun 1997	Medium (not re-fetched)
RUGD and RELLIS-3D are unstructured outdoor datasets	Your reference list	RUGD IROS 2019; RELLIS-3D repo	Medium; class mapping to our 7 classes unverified
ORB-SLAM3 supports visual-inertial operation	Your reference list	Campos et al. 2020, arXiv 2007.11898	Medium (not re-fetched)
$3 \times \tau_{\text{stop}}$ can exceed a fixed 3 s horizon when $\tau_{\text{stop}} > 1$ s	Arithmetic on the blueprint's own rules	Section 7	High (derived)
Full benchmark is roughly 1,000–1,500 runs	25 runs $\times$ 10 scenarios $\times$ 5 configurations $\approx$ 1,250	Arithmetic	High (count); wall-clock Low (ESTIMATE)
Softmax outputs tend to be over-confident under domain shift	General finding, not measured here	Unverified here	Medium; measure ECE yourself
Gazebo can assign per-surface friction	Expected SDF capability	Verify in Harmonic docs before relying on it	Low until checked
All performance numbers in Section 21	None yet	—	PROPOSED target

27. FINAL ARCHITECTURE

VISTRA = blueprint core + three small additions, nothing else.

1. **Blueprint core, unchanged:** quality monitor → SegFormer-B0 → ground projection → BEV risk map (entropy + geometry + pose inflation) → stereo VO + EKF → D* Lite + DWA → TTC → NCS governor → recovery → explainer and dashboard.
2. **Fix:** prediction horizon scales with stopping time; 7th class soft/mud; negative obstacles handled through the geometry term, measured separately.
3. **Addition (the only PRAGYA module kept):** ECT in simulation with a hard week-6 cut-rule.
4. **Process change:** two-tier simulation; baseline harness built in weeks 1-3; every number labelled TARGET until measured.

Cut for good: Nav2 as the planner, Jetson/TensorRT as a dependency, energy-aware planning, Cityscapes/KITTI/BDD datasets, Monte-Carlo dropout, YOLO-seg for terrain, patent claims.

28. 30-SECOND PITCH

Most off-road robots trust their cameras completely and fail silently when the camera or the pose estimate is wrong. VISTRA carries doubt from pixels to wheels: uncertain ground becomes costly, shaky localization makes the map more conservative, and one confidence score slows, recovers or stops the vehicle before it hits anything. It learns from its own wheel slip which ground is really safe, and it explains every decision live. Then we prove it: same seeds, a conventional baseline, ablations, and a judge button that breaks the camera on demand.

29. 2-MINUTE PITCH

Problem (20 s). BEL's PS asks for GPS-free, camera-primary navigation for a UGV.

Outdoors, the ground has no lanes, a rock detector does not tell you whether soft soil will trap the wheels, and vision itself degrades with blur, darkness and glare. A typical stack of a detector, SLAM and A* treats its outputs as truth.

Idea (30 s). VISTRA treats uncertainty as the primary signal. We estimate what is confidently drivable, not just what is an obstacle. Segmentation entropy raises risk; pose covariance raises it further; one weakest-link confidence score sets the speed and mode. A predictive collision layer compares time-to-collision with the real stopping time, including our measured latency.

Differentiators (30 s). First, uncertainty flows end to end. Second, behaviour is gated by confidence, with explicit recovery for blur, darkness, camera cover and lost localization. Third, the vehicle calibrates its terrain scores from simulated slip and vibration, so a patch that looks safe but is not gets corrected after one crossing. Every decision appears in plain language on the dashboard.

Proof (25 s). We do not show one lucky run. We run the same seeded scenarios for a conventional baseline and for VISTRA, with confidence intervals and ablations, and report where the baseline wins. Judges can inject an obstacle, blur, darkness or a covered camera live.

Honesty and next step (15 s). Closed-loop results are simulated; perception is validated on real outdoor footage. A physical UGV is the next validation step, and the same code runs against it through one interface.

30. JUDGE Q&A

1. *Isn't this just YOLO + SLAM + A?** That is our baseline. VISTRA differs in uncertainty-propagating costs, confidence-gated behaviour and latency-aware TTC, each with an ablation on identical seeds.
2. **You use IMU and wheel odometry. Is it camera-based?** Vision is the primary perception and localization sensor. IMU and wheels bridge short dropouts and give the learning signal. A camera-only ablation reports the drift difference.
3. **Does it work in the real world?** Perception, mapping and VO run on real footage open-loop. Closed-loop navigation is validated in simulation. A physical UGV is the next step.
4. **How do you handle ditches?** Poorly from appearance alone. Uncertain ground is costly by default, stereo geometry adds a second cue, and we report ditch recall separately.
5. **Why not end-to-end learned planning?** It is hard to verify, explain or bound for safety, and needs far more data. Our planners are classical and inspectable.
6. **How did you tune thresholds?** From seeded runs on a tuning seed set, then evaluated on separate seeds. Final values and sensitivity are reported.
7. **What is your latency budget?** Target end-to-end under 150 ms (PROPOSED), measured on our hardware, and the measured value feeds τ_{stop} . We report the real number.
8. **What are your limitations?** Flat-ground projection, sim-to-real gap, weak ditch recall, hand-set weights, sim-only learning loop. We state them first.

9. **Is entropy a real uncertainty measure?** It is a proxy. We apply temperature scaling and report calibration error; it is not claimed as epistemic uncertainty.
10. **What if entropy makes narrow trails look unsafe?** We weight it by local confidence, tune it, and show the entropy-off ablation so the net effect is visible.
11. **Why weakest-link rather than an average?** An average lets one failed subsystem hide behind healthy ones. We smooth and calibrate components to avoid permanent caution, and measure the time cost.
12. **How do you prevent state flicker?** Different enter and exit thresholds, plus temporal smoothing.
13. **What is new compared with TerraPN?** TerraPN already learns terrain cost from vibration and odometry error. Our contribution is combining it with pose-aware risk, a confidence governor and a latency-aware safety layer, and evaluating the whole with ablations.
14. **Isn't your learning loop circular in simulation?** We randomise hidden friction independent of visual class, so the loop corrects a wrong visual prior. It still uses simulated physics, and we say so.
15. **Why SegFormer-B0?** About 3.7–3.8 M parameters, easy fine-tuning, good accuracy per parameter. We benchmark FPS ourselves and do not quote paper numbers.
16. **Why not TensorRT or Jetson?** Not needed to demonstrate the logic; ONNX Runtime on CPU suffices for the prototype. It is a deployment path if FPS demands it.
17. **Why write your own planners instead of Nav2?** Explainability and tuning speed. Nav2 is our baseline.
18. **How is global target defined without GPS?** Relative coordinates in a local frame initialised at start, a clicked point on the local map, or a marker. VISTRA navigates in that frame.
19. **What happens if localization is lost?** Stop or creep, relocalize against recent keyframes, retrace toward the last good keyframe, dead-reckon with growing covariance, and safe-stop on timeout.
20. **What if the camera is covered?** Frame-variance collapse is detected; the vehicle stops after a short grace period and alerts.
21. **How do you avoid sim-to-real overfitting?** Mixed training, sequence-level splits, and a held-out real set that never enters training.
22. **How many test runs and how do you report them?** About 25 seeded runs per scenario per configuration, with means and confidence intervals. Individual runs are not evidence.

23. **What if the baseline wins a scenario?** We report it and explain why. That is a stronger position than hiding it.
24. **How does the dashboard help an operator?** It shows risk, confidence components, TTC and a plain-language reason for every state change.
25. **What about moving obstacles?** Constant-velocity prediction from tracked detections. We acknowledge it fails for erratic motion, so the TTC layer keeps margin.
26. **How do you handle slopes?** IMU pitch/roll compensation and down-weighting of far cells in the projection; we state the flat-ground limit.
27. **Cybersecurity?** Local-only autonomy, token-protected dashboard, signed models in prototype; authenticated channel and hardware e-stop for deployment.
28. **What does it cost?** Prototype is software-only. Optional physical validation is an ESTIMATE of roughly ₹35,000–90,000.
29. **How does it scale to a fleet?** Autonomy stays on-vehicle; fleet layers add telemetry, model registry and staged updates. Shared maps are Future work.
30. **What would you build next?** Physical UGV trials, patch-appearance learning for ECT, ensemble or test-time uncertainty, and field data collection.

31. FINAL IMPLEMENTATION CHECKLIST

Before anything else (this week)

- ☐ Fix the PRAGYA-Nav leftovers, the D* Lite asterisk and the Nav2/TensorRT stack lines in the NEXORAS final deck; confirm the screening deck's page numbering.
- ☐ Decide once: RTAB-Map or ORB-SLAM3; Gazebo Harmonic (Webots fallback); freeze both.
- ☐ Verify the 7-class mapping against RUGD/RELLIS label lists.
- ☐ Read the WayFASTER and TerraPN papers directly before citing them in any slide.

Build

- ☐ Docker/Ubuntu 24.04 image shared by the team.
- ☐ Hardware abstraction layer and sim vehicle; seeded scenario generator (including hidden friction).
- ☐ Baseline stack and benchmark harness (weeks 1–3).
- ☐ SegFormer-B0 fine-tuned, exported to ONNX; per-class IoU, FPS, latency, ECE recorded.
- ☐ BEV risk map, temporal fusion, pose-covariance inflation, geometry term.

- ☐ Stereo VO + EKF with ATE logged; camera-only ablation.
- ☐ D* Lite + DWA + wheel mapper; A→B on a clear world.
- ☐ TTC layer with $H = \max(3 \text{ s}, 3 \times \tau_{\text{stop}} + 0.5 \text{ s})$ and measured latency.
- ☐ NCS, hysteresis FSM, Recovery Manager, explanation feed.
- ☐ ECT with week-6 cut-rule.
- ☐ Dashboard with all panels and fault-injection buttons.

Evidence

- ☐ 20–30 seeded runs × 10 scenarios × baseline + VISTRA + ablations (Tier 1); Tier 2 showcase runs.
- ☐ Report collisions, clearance, false-stop rate, success, ATE, recovery time, FPS, latency, with confidence intervals.
- ☐ Held-out real test set evaluated once, at the end.

Submission and demo

- ☐ Scope frozen at end of week 6; nothing new afterwards.
- ☐ Every number in the deck is MEASURED or labelled TARGET/ESTIMATE.
- ☐ Limitations slide written before judges ask.
- ☐ Backup: rosbag replays, full-run video, recorded-video mode, playback dashboard; rehearsed three times.
- ☐ Repo, MVP link and deck all show the same name and the same architecture.